SCambrian – startup guide

From a clean clone on ORFEO to the first results (six-step pipeline, November 2026)

1. The picture

SCambrian takes protein FASTA sequences of several **samples**. It clusters them by **sequence** (MMseqs2) and by **representation** (ESMC embeddings projected with UMAP and t-SNE). Each clustering is run twice: on every target alone (**internal**) and on all samples together (**joint**). The result is **one summary table**: one row per sequence, one column per clustering.

Each sample is a **target** (a proteome you study) or a **probe** (a set of known proteins, e.g. Insecta immune proteins). A probe is clustered with the targets but never counts as a species: it only shows which target proteins co-cluster with it.

Every command below is run from the repository root.

```
SCambrian/                      (git clone)
  esmc_config.sh                builds ./.env once
  configs/query_configs.tsv     WHAT to analyse (one row per sample)
  configs/param_configs.yaml    HOW (one section per Python script)
  slurms/run_all.sh             Snakemake driver: submits only the steps with missing or stale outputs
  src/                          the Python code of each step
  data/                         per-sample data, shared across analyses (not in git)
  analyses/<analysis_name>/     one folder per analysis (not in git)
  logs/<step>/                  one log per SLURM job
```

2. Install (once per cluster account)

```
git clone https://github.com/ssenci97/SCambrian.git
cd SCambrian
sbatch esmc_config.sh
tail -f logs/setup_ESMC.log
```

The job builds the conda env `./.env` (Python 3.10, PyTorch 2.4.1 cu121, flash-attn, ESM, Scanpy). It succeeded when the log ends with `All pipeline dependencies successfully verified!`. Running it again never changes an existing `./.env`; it only re-checks it.

MMseqs2 is **not** installed by the job: it must be on the `PATH`, or its path must be set in `preprocess_sequences.mmseqs_executabl` and `run_mmseqs2.mmseqs_executable`.

To use the env in a shell:

```
source ~/scratch/miniconda/etc/profile.d/conda.sh
conda activate ./.env
```

3. Configure your analysis

`configs/query_configs.tsv`: one tab-separated row per sample, with no comments.

- `label`: short, **unique** name used in every file name, e.g. `aegypti`

- `query`: a UniProtKB query, e.g. `(organism_id:7159)` (sequences and annotation table are downloaded), or the path of a local FASTA file ending in `.fa`, `.faa` or `.fasta` (copied)
- `taxonomic_ref`: name shown in figures, e.g. `Ae. aegypti`
- `input_type`: `target` (the default) or `probe`

FASTA ids (the first word of each header) must be unique across samples; step 02 stops otherwise.

`configs/param_configs.yaml`: one section per Python script. The keys you will most often change:

- `preprocess_sequences.analysis_name`: `null` names the analysis `{YYYY-MM-DD}_SCambrian`
- `preprocess_sequences.min_length`, `dedup_min_identity`
- `run_mmseqs2.cluster_identity_thresholds`
- `extract_reps_biohub.models`: `esmc_300m` (default), `esmc_600m`, `esmc_6b`
- `npz_scanpy.leiden_resolution`, `knn_neighbors`
- `h5ad_scanpy.report_series` and its thresholds for the species and probe report

4. Run

```
DRY_RUN=1 sbatch slurms/run_all.sh      # list the jobs that would be submitted
sbatch slurms/run_all.sh                # run every step that is missing or stale
sbatch slurms/run_all.sh 05 06          # target some steps (and what they still need)
ANALYSIS_NAME=test_SCambrian sbatch slurms/run_all.sh 02
squeue -u $USER                         # follow the jobs; logs in logs/snakemake/ and logs/<step>/
```

`run_all.sh` is a small THIN job running Snakemake. Snakemake submits each step script, one array task at a time, only when its outputs are missing, older than their inputs or built with other parameters, and only after the steps it needs succeeded. CPU steps go to the free node with the most idle CPUs and memory (`slurms/best_node.sh`); step 04 always goes to DGX. The dry run is also the quickest check of the configs: it reads both files and lists the jobs and the analysis name.

The analysis name is fixed once and passed to every job. It is taken from `ANALYSIS_NAME`, else from the YAML, else it is today's `{YYYY-MM-DD}_SCambrian` when step 02 is requested (or no analysis exists yet), or the newest `analyses/*_SCambrian` folder, so a plain `run_all.sh` resumes the latest analysis.

step	does	output
01	UniProtKB download or local FASTA copy	`data/sequences/`, `data/features/`
02	validate, tag (`sc_label=`, `sc_type=`), drop probe copies of targets, dedup per sample	`data/preprocessed/`, `analyses/<A>/sc_metadata.tsv`
03	MMseqs2 clusters at several identities, internal and joint	`data/clusters/`, `analyses/<A>/mmseqs_joint/`
04	ESMC embeddings (GPU)	`data/esm-reps/<model>/`
05	UMAP and t-SNE (2D, 3D), internal and joint; Leiden on every layout	`analyses/<A>/<model>/projections.h5ad`
06	cluster names, summary, species report, 3D viewer, RO-Crate	`analyses/<A>/<model>/`

5. Read the results

Every cluster has one name:

```
{analysis_name}_{u|t|s}_{i|j}_{stat}_{SERIAL}

u / t / s   UMAP layout / t-SNE layout / sequence (MMseqs2)
i / j       internal (one target alone) / joint (all samples together)
stat        leiden2d, leiden3d, or the MMseqs2 identity id90, id50, ...
SERIAL      internal: <label>_001, _002, ...   joint: a, b, ..., aa, ...
            (by decreasing size; a joint cluster is UPPER case, A, B, ...,
```

when it holds probe and target proteins together)

For example, `2026-11-02_SCambrian_u_j_leiden2d_C` is the third-largest Leiden cluster of the joint 2D UMAP, and it holds probe proteins.

In `analyses/<A>/<model>/`:

- `sc_summary.tsv`: **the main result**. The kept rows of `sc_metadata.tsv`, plus one column per series; each cell holds the SERIAL.
- `sc_joint_clusters.tsv`: per joint cluster, its size and the count and percent of every sample, including the probe percent.
- `sc_*.png`: cluster composition, clusters per series.
- `<model>_scanpy.pdf`: species report of `h5ad_scanpy.report_series`.
- `probe_cluster_enrichment__<series>.csv`, `probe_capture__<series>.csv`: the probe clusters (UPPERcase clusters significantly enriched in a probe) and the percent of every target inside them.
- `interactive/views_3d.html`: 3D viewer (open it in a browser).

`analyses/<A>/sc_metadata.tsv` lists **every** input record with its `status`: `kept`, `invalid`, `probe_overlap` or `duplicate`.

The notebooks in `nbs/` load every stage; use `./.env` as the kernel and set `ANALYSIS_NAME` and `MODEL` at the top.

6. Changes and reruns

- After changing a config or adding a sample, run `sbatch slurms/run_all.sh` again: only missing or outdated outputs are rebuilt, and a new sample is embedded alone. To force a step, set `overwrite: true` in its section.
- Data in `data/` (downloads, dedup, embeddings) is reused by every analysis; a new analysis name only rebuilds the joint steps and the summary.
- Outputs of the previous layout are not deleted automatically: `bash slurms/list_relics.sh` prints them; remove them by hand.

Full details are in `README.md`.